File Guide

What every file in the project does, folder by folder.

This guide explains every file in the project and the job it does. The project follows the standard Spring Boot layered structure: a request travels Controller → Service → Repository → Database, and the answer travels back out the same way.

The big picture

```
Browser (static/ HTML+CSS+JS)
| HTTP + JSON
v
Controller -> Service -> Repository -> PostgreSQL
(HTTP) (rules, (SQL via
locking) Spring Data)
```

Each layer has one responsibility. That separation is what keeps the code testable and is required non-functional requirement NFR-04.

Project root

File	What it does
pom.xml	Maven build file. Lists the libraries (Spring Boot, JPA, PostgreSQL driver, test tools) and how to package the app into a runnable jar.
README.md	Short overview and quick-start for the repository.
application.properties	Main settings: database connection, JPA behaviour, connection-pool size, and the advisory-lock on/off flag.

model/ — the data (entities)

Plain Java classes that map one-to-one to database tables. JPA turns objects into rows and back.

File	What it does
User.java	A person who books. Maps to the users table.
Resource.java	A bookable thing (room, doctor, pod). Holds capacity — how many confirmed bookings may overlap.
Booking.java	The core entity: links a user to a resource for a time window [start, end). Maps to the bookings table.
BookingStatus.java	An enum: CONFIRMED or CANCELLED. Only CONFIRMED bookings occupy a slot.

repository/ — database access

Interfaces that extend Spring Data JpaRepository. Spring writes the SQL for the standard methods automatically; we add a few custom queries.

File	What it does
UserRepository.java	CRUD for users.
ResourceRepository.java	CRUD for resources.
BookingRepository.java	CRUD plus the two queries that matter: **countOverlapping** (how many confirmed bookings clash with a window) and **acquireResourceLock** (the PostgreSQL advisory lock).

service/ — the business logic

File	What it does
BookingService.java	The heart of the project. Its createBooking method runs in one transaction: take the advisory lock, count overlaps, reject if at capacity, otherwise insert. This is where double-bookings are prevented. Also handles cancel and lookups.
ResourceService.java	Read-only logic: list resources, and build the hourly slot grid for a day marking each slot available or full.

> **Note:** The concurrency fix lives here. The advisory lock serialises competing bookings for the same resource, so the check-then-insert can never be interrupted half-way — even for a slot that has no rows yet, which is where plain SELECT ... FOR UPDATE would fail.

controller/ — the web layer (REST API)

File	What it does
BookingController.java	Endpoints to create, fetch, cancel bookings and list a user's bookings. Returns 201 / 409 / 404 as appropriate.
ResourceController.java	Endpoints to list resources and get a resource's available slots for a date.
UserController.java	Lists users so the demo UI can offer a 'book as' picker.
GlobalExceptionHandler.java	One place that turns exceptions into clean JSON errors with the right HTTP status (conflict → 409, not found → 404, bad input → 400).

dto/ — request & response shapes

Data Transfer Objects define exactly what JSON goes in and out, so the API never exposes the raw database entities.

File	What it does
BookingRequest.java	The incoming JSON for a new booking (userId, resourceId, startTime, endTime) with validation.
BookingResponse.java	The JSON returned for a booking.
ResourceResponse.java	The JSON shape of a resource in listings.
SlotResponse.java	One time slot with its availability and booked/capacity counts.
ErrorResponse.java	A consistent { error, message } shape for failures.

exception/ — custom errors

File	What it does
BookingConflictException.java	Thrown when a slot is full; becomes HTTP 409 and rolls the transaction back.
ResourceNotFoundException.java	Thrown for a missing id; becomes HTTP 404.

config/ — startup helpers

File	What it does
DataSeeder.java	Inserts a few sample users and resources on first launch (only if the tables are empty) so the UI has data to show.
BookingApplication.java	The main class. Starts Spring Boot and the embedded web server. (Lives in the project's base package.)

static/ — the frontend

Served by the backend at http://localhost:8080. Plain web files, no framework, so they are easy to read.

File	What it does
index.html	The page structure: resource cards, the slot grid, and the user's bookings list.
styles.css	All the visual styling (the warm 'paper' theme, cards, slot colours, toast messages).
app.js	The logic: calls the REST API with fetch(), renders resources and slots, books and cancels, and shows success/conflict toasts.

db/ — SQL scripts

File	What it does
schema.sql	The full table definitions, for reference or manual setup. (The app also creates these automatically.)
hardening.sql	Optional EXCLUDE constraint — the database-level safety-net for capacity-1 resources.

jmeter/ — load testing

File	What it does
booking-stress-test.jmx	Apache JMeter plan. Scenario 1 fires 50 identical bookings simultaneously (expect 1 success, 49 conflicts). Scenario 2 fires 100 unique bookings (expect all to succeed).

src/test/ — automated tests

File	What it does
BookingServiceTest.java	Unit tests with mocked repositories: a free slot books, a full slot is rejected, capacity > 1 allows multiple bookings, bad time ranges are refused.
BookingIntegrationTest.java	Full-stack tests on an in-memory H2 database via MockMvc: 201 for a valid booking, 409 for a conflict, adjacent slots allowed, cancelling frees the slot.
application-test.properties	Points the tests at H2 and turns the PostgreSQL-only advisory lock off.